CURSO PARA PRINCIPIANTES · NÃO É PRECISO NENHUM CONHECIMENTO PRÉVIO

Criar um site de A a Z

Perceber como um site verdadeiro é construído, publicado e gerido no dia a dia — abrindo, ficheiro a ficheiro, o de **The Evergreen Project**.

Site: dimogdor.github.io/Evergreen · Código-fonte: github.com/Dimogdor/Evergreen

COMO LER ESTE CURSO

Cada capítulo explica uma ideia de forma simples e depois mostra **onde a ver no site** (caixa verde «Ver») e por vezes propõe um pequeno **exercício** (caixa laranja). As caixas azuis «Para dizer» são frases-chave para a explicação em voz alta. Não é preciso decorar tudo: o objetivo é perceber *quem faz o quê*.

Índice

1. Como funciona um site
2. As três linguagens da web: HTML, CSS, JavaScript
3. A caixa de ferramentas
4. O repositório GitHub: a pasta do projeto
5. Astro: o «gerador» do site
6. Separar o conteúdo do código (YAML)
7. O design na prática: da ideia ao ecrã
8. Paginação e design adaptável (telemóvel, tablet, computador)
9. Fotografias e vídeos
10. Um site em 8 línguas
11. As funcionalidades, uma a uma
12. A área de administração (sem código)
13. O pequeno servidor: o Worker
14. Ser encontrado no Google (SEO)
15. Acessibilidade: um site para todos
16. Segurança
17. Rapidez (desempenho)
18. Testar o site
19. Publicar: alojamento, domínio, DNS
20. O ciclo de vida de uma alteração
21. Por onde começar a aprender
22. Glossário

PARA DIZER NA INTRODUÇÃO

«Um site não é mais do que ficheiros de texto que um navegador sabe ler. Tudo o resto — as ferramentas, os serviços — serve para escrever esses ficheiros mais depressa, torná-los bonitos e pô-los online.»

1 · Como funciona um site

Quando escrevemos um endereço num navegador (Chrome, Safari, Firefox…), acontecem três coisas:

Navegador → **pede** → Servidor (um computador sempre ligado) → **devolve ficheiros** → O navegador mostra-os

1. **O endereço** (URL), p. ex. `https://dimogdor.github.io/Evergreen/pt/contact/`, indica que servidor contactar e que página pedir.
2. **O servidor** devolve ficheiros: uma página **HTML**, folhas de estilo **CSS**, scripts **JavaScript**, imagens.
3. **O navegador** junta tudo e mostra a página.

Site estático ou dinâmico?

	Site estático (o nosso caso)	Site dinâmico (p. ex. WordPress)
Princípio	As páginas são feitas **antecipadamente**, uma vez, e depois simplesmente enviadas.	Cada página é feita **a cada visita** por um programa e uma base de dados.
Velocidade	Muito rápido	Mais lento
Segurança	Muito poucas falhas possíveis	Atualizações de segurança frequentes
Custo	Alojamento gratuito	Normalmente um servidor pago

O site da Evergreen é **estático**: depois de cada alteração, uma ferramenta faz todas as páginas (171!) em poucos segundos e depois são colocadas no alojamento. Veremos mais à frente como temos, ainda assim, uma área de administração e formulários.

> **VER**
> Abra o site e clique com o botão direito → **«Ver código-fonte da página»**: é o HTML que o servidor enviou. Depois botão direito → **«Inspecionar»**: são as **ferramentas de programador** do navegador, o melhor amigo de quem cria sites.

2 · As três linguagens da web

Todos os sites do mundo assentam em três linguagens, cada uma com um papel preciso. Uma imagem simples: uma casa.

Linguagem	Papel	Na casa
HTML	A **estrutura** e o conteúdo: títulos, parágrafos, imagens, ligações, formulários.	As paredes e as divisões
CSS	O **aspeto**: cores, tipos de letra, tamanhos, espaçamentos, disposição.	A pintura e a decoração
JavaScript	O **comportamento**: o que reage quando se clica, o que muda sem recarregar.	A eletricidade, os interruptores

HTML: «etiquetas»

```
<h1>Contacto</h1>                      ← um título grande
<p>Escreva-nos, respondemos depressa.</p>  ← um parágrafo
<a href="mailto:evergreenecovillage@gmail.com">Escrever-nos</a>  ← uma ligação
<img src="tenda.webp" alt="Uma tenda na floresta">           ← uma imagem + a sua descrição
```

CSS: «regras»

```
h1 { font-family: 'EB Garamond'; color: #22402a; }   ← todos os títulos grandes em verde-escuro
.btn { border-radius: 999px; padding: 0.85em 1.4em; } ← botões arredondados
```

JavaScript: «instruções»

```
botao.addEventListener('click', () => {                ← quando se clica no botão…
  navigator.clipboard.writeText('GR7701103910000039100269800');  ← …copiar o IBAN
});
```

VER NO REPOSITÓRIO

- CSS: `src/styles/global.css` — todo o estilo do site num único ficheiro.
- JavaScript: no fim de `src/pages/[lang]/[page].astro`, o botão «Copiar» do IBAN (as linhas `<script>`).
- HTML: em todos os ficheiros `.astro` (ver capítulo 5).

E o TypeScript?

É JavaScript com «etiquetas» que indicam o tipo de cada dado (texto, número…). Evita erros. Os ficheiros `.ts` do projeto (p. ex. `src/calendar.ts`) estão escritos assim.

PARA DIZER

«Começa pelo HTML e pelo CSS: em duas semanas consegues fazer uma página a sério. O JavaScript vem depois, quando quiseres que a página reaja.»

3 · A caixa de ferramentas

Ferramenta	Para que serve	Gratuita?
Um editor de código: VS Code	Escrever e organizar os ficheiros, com cores e sugestões.	Sim
Um navegador + as suas ferramentas de programador (F12)	Ver o resultado, inspecionar, testar num «falso» telemóvel.	Sim
O terminal	Escrever comandos (arrancar o site, instalar ferramentas).	Sim
Node.js e **npm**	O Node corre JavaScript no computador; o npm instala «pacotes» (ferramentas escritas por outras pessoas).	Sim
Git	Guardar o histórico de cada alteração (como «Anular», mas para sempre).	Sim
GitHub	Guardar o projeto online, colaborar, lançar a publicação automática.	Sim
Figma (opcional)	Desenhar maquetas antes de programar.	Sim (plano básico)

Os três comandos do projeto

```
npm install     ← descarrega as ferramentas de package.json (uma só vez)
npm run dev     ← arranca o site no computador: http://localhost:4321 (atualiza a cada gravação)
npm run build   ← faz a versão final na pasta dist/
```

VER

`package.json`: o «cartão de cidadão» do projeto. Mostra os comandos (`scripts`) e os pacotes usados (`dependencies`): **astro** (o gerador), **@astrojs/sitemap** (mapa do site para o Google), **marked** (transforma texto simples em HTML), **yaml** (lê os ficheiros de conteúdo) e dois tipos de letra (**Inter** e **EB Garamond**).
`.node-version`: a versão do Node a usar (22).

Git em 5 palavras

- **Repositório**: a pasta do projeto com todo o seu histórico.
- **Commit**: uma «fotografia» dos ficheiros num momento, com uma mensagem («Inhabitants: new photo of our dogs»).
- **Ramo** (branch): uma cópia de trabalho paralela para experimentar sem estragar a versão oficial (`main`).
- **Pull request**: o pedido para juntar um ramo ao `main`, com revisão.
- **Push / pull**: enviar / ir buscar os commits para / do GitHub.

EXPERIMENTAR

Em github.com/Dimogdor/Evergreen, clique em **«Commits»**: vê cada alteração feita ao site, quem a fez e quando, e pode mostrar exatamente as linhas modificadas.

4 · O repositório: a planta do projeto

Eis como as pastas estão organizadas. Arrumar bem é metade do trabalho.

```
Evergreen/
├── package.json            ← cartão de cidadão, comandos, ferramentas
├── astro.config.mjs        ← definições do gerador (endereço do site, segurança)
├── README.md               ← manual do projeto
├── docs/GUIDE-ADMIN.md     ← manual para quem administra
├── public/                 ← ficheiros copiados tal como estão
│   ├── admin/              ← a área de administração (config.yml, index.html)
│   ├── uploads/videos/     ← vídeos
│   └── _headers            ← regras de segurança/cache para o alojamento
├── src/                    ← o coração do site
│   ├── pages/              ← uma página = um ficheiro (capítulo 5)
│   ├── layouts/Base.astro  ← a moldura comum: cabeçalho, menu, rodapé
│   ├── components/         ← peças reutilizáveis (imagem, lista de eventos…)
│   ├── styles/global.css   ← todo o design
│   ├── content/            ← TODO o texto do site, em ficheiros YAML (capítulo 6)
│   ├── assets/uploads/     ← fotografias originais (otimizadas automaticamente)
│   ├── lib.ts              ← pequenas funções úteis (ler conteúdo, ligações…)
│   ├── i18n.ts             ← as línguas
│   └── calendar.ts         ← datas, fusos horários, ficheiros de agenda
├── worker/                 ← o pequeno servidor (acesso admin, histórias, ajuda à escrita)
├── tests/                  ← verificações automáticas
└── .github/workflows/      ← robôs do GitHub: publicação, compressão de vídeo
```

PARA DIZER

«O código (páginas, design) e o conteúdo (textos, fotografias) estão em pastas separadas. Quem gere o lugar só mexe no conteúdo, pela área de administração; o código quase nunca muda.»

5 · Astro: o «gerador» do site

Escrever 171 páginas HTML à mão seria impossível (8 línguas × umas vinte páginas). Por isso usamos um **gerador de sites estáticos**: escrevemos **um modelo** por tipo de página e a ferramenta preenche-o com cada conteúdo, em cada língua. Aqui é o **Astro** (astro.build).

Um ficheiro .astro = duas partes

```
---
// Parte 1, entre os traços: JavaScript que prepara os dados (corre durante a construção do site)
const page = await one('pages', 'contact', lang);
---
<!-- Parte 2: HTML, onde os dados são inseridos entre chavetas -->
<h1>{page.title}</h1>
<p class="lead">{page.intro}</p>
```

As páginas = os endereços

Em `src/pages/`, o nome do ficheiro torna-se o endereço. Os parênteses retos `[ ]` querem dizer «variável»:

Ficheiro	Endereços produzidos
`src/pages/[lang]/index.astro`	/el/, /en/, /pt/… (as 8 páginas iniciais)
`src/pages/[lang]/[page].astro`	/pt/contact/, /pt/place/, /pt/support/… (todas as páginas de texto)
`src/pages/[lang]/activities/[slug].astro`	/pt/activities/retreats/… (uma página por atividade)
`src/pages/[lang]/calendar.astro`	/pt/calendar/
`src/pages/[lang]/gallery.astro`, `stories.astro`	a galeria, as histórias
`src/pages/calendar.ics.ts`	/calendar.ics: um ficheiro de agenda, não uma página!

O layout: a moldura comum

`src/layouts/Base.astro` contém o que é igual em todas as páginas: o `<head>` invisível (título do separador, descrição para o Google, ícone), o cabeçalho com o logótipo e o menu, a escolha de língua, o rodapé com os contactos. Cada página «encaixa» lá dentro com `<Base> … </Base>`.

Os componentes: peças de LEGO

O mais simples para perceber: abra `src/components/Story.astro` (12 linhas) — uma fotografia, uma citação (`<blockquote>`), o nome e o país.

Componente	O que faz
`src/components/Img.astro`	Mostra qualquer fotografia, otimizada automaticamente (capítulo 9).
`Blocks.astro`	As secções «título + texto + fotografia» que alternam esquerda/direita.
`EventList.astro`	A lista de eventos com datas, preços e botão «Adicionar à agenda».
`Story.astro`	Um cartão de história.
`Video.astro`	Um vídeo do YouTube/Vimeo que só carrega quando se clica.

6 · Separar o conteúdo do código

Todos os textos, datas, preços e caminhos de fotografias estão em `src/content/`, no formato **YAML**: um formato de texto muito legível, feito de linhas «chave: valor» e de recuos.

> **VER: UM EVENTO**
>
> `src/content/events/essence-of-yoga-2026.yml`

```
el:                                  ← a versão grega (língua principal)
  start: '2026-11-28'                ← data de início
  end: '2026-12-05'                  ← data de fim
  category: retreat
  image: /src/assets/uploads/yin-shala.webp
  link: https://bookretreats.com/r/8-day-essence-of-yoga-…
  title: Essence of Yoga — χειμερινό εντατικό …
  price: 380 € early bird (κράτηση έως 28/10/2026) · 450 € κανονική τιμή
pt:                                  ← em português: só o que se traduz
  title: Essence of Yoga — imersão de inverno com Panos Katsandris
  price: 380 € early bird (reserva até 28/10/2026) · 450 € preço normal
```

Pasta	Conteúdo
`settings/site.yml`	Nome, e-mail, telefone, redes, IBAN, cores do site
`settings/ui.yml`	Os pequenos textos da interface (menu, botões) nas 8 línguas
`pages/`	O texto de cada página (início, o lugar, modo de vida…)
`programs/`	As atividades (voluntariado, retiros…)
`events/`	A agenda: um ficheiro por evento
`inhabitants/`	Os habitantes: Katerina, Kostas, os cães, os gatos, as galinhas…
`albums/`, `testimonials/`, `extra/`	Galeria, histórias, páginas criadas na administração

Como é que o código lê este conteúdo? `src/content.config.ts` declara as pastas, e a função `list()` de `src/lib.ts` lê-as e escolhe a língua certa.

> **PARA DIZER**
>
> «Sem base de dados: o conteúdo são simples ficheiros de texto arrumados em pastas. Podem ser lidos, guardados, pode-se voltar atrás. A administração apenas edita estes ficheiros por nós.»

7 · O design na prática: da ideia ao ecrã

Passo 1 — Juntar inspirações («moodboard»)

Antes de programar, olhamos para sites de que gostamos e anotamos *porquê*. Para a Evergreen: kaleidoscope.media (títulos grandes e elegantes, muito espaço, secções numeradas «01 —») e evergreenyogastudios.com (a identidade da família Evergreen). Também juntamos o material: logótipo, fotografias recentes, tom dos textos.

Passo 2 — Escolher uma paleta

Bastam 4 a 6 cores, tiradas do tema: aqui papel creme, musgo, floresta profunda, terracota. Verificamos o **contraste** (texto legível sobre o fundo) com uma ferramenta como *webaim.org/resources/contrastchecker*.

```
:root {                       ← src/styles/global.css, logo no topo
  --paper: #f4f0e6;           fundo creme
  --ink: #17231a;             texto quase preto, com um toque de verde
  --moss: #3d673f;            verde-musgo
  --moss-deep: #22402a;       verde-floresta (faixa CES, rodapé)
  --clay: #b0622a;            terracota (toques de destaque)
}
```

Estas «variáveis» (`--paper`…) são reutilizadas em todo o lado; mudar uma linha muda o site inteiro. Pode até ser mudado na administração (Definições → Cores).

Passo 3 — Escolher dois tipos de letra

Um **tipo de letra com serifa** para os títulos (EB Garamond: caloroso, «de livro») e um **tipo de letra simples** para o texto (Inter: muito legível no ecrã). Vêm do Google Fonts mas estão **instalados no projeto** através dos pacotes `@fontsource-variable/…`: nenhuma chamada ao Google, mais rápido e mais respeitador da privacidade.

Passo 4 — Uma escala de espaçamentos e tamanhos

Em vez de tamanhos fixos usamos `clamp(mín, ideal, máx)`: o título cresce com o ecrã sem nunca ficar pequeno ou grande demais. Exemplo: `--gutter: clamp(16px, 4vw, 56px)` = a margem lateral.

Passo 5 — «Componentes visuais» coerentes

Botões redondos em maiúsculas espaçadas (`.btn`), pequenas etiquetas (`.label`), cartões fotografia + título (`.card`), linhas horizontais numeradas (`.rule`). Uma vez definidos, reutilizam-se: o site parece «desenhado», não remendado.

Passo 6 — Maqueta (opcional) e depois código

No Figma desenhamos a página inicial em versão computador e telemóvel. Depois traduzimo-la em HTML (estrutura) e CSS (estilo) — e afinamos diretamente no navegador com as ferramentas de programador.

> **EXPERIMENTAR**
>
> Abra o site, F12 → separador «Elementos», clique em `<html>` e mude `--moss-deep` para `#5a2a6e` : o site inteiro muda de cor em direto (nada se estraga, basta recarregar).

8 · Paginação e design adaptável

O mesmo site tem de ficar bem num telemóvel (390 px de largura) e num ecrã grande (1440 px ou mais). É o **design adaptável** (responsive).

As duas ferramentas de paginação do CSS moderno

Ferramenta	Para quê	Exemplo no site
Flexbox	Alinhar elementos numa linha (ou numa coluna).	O cabeçalho: logótipo à esquerda, menu à direita (`.site-header`).
Grid	Uma grelha de linhas e colunas.	Os cartões de atividades, os habitantes (`.grid.cols-3`, `.cols-4`).

```
.cols-3 { grid-template-columns: repeat(auto-fill, minmax(min(100%, 340px), 1fr)); }
← «tantas colunas de pelo menos 340 px quantas couberem»: 3 no computador, 1 no telemóvel, sem mais nada

@media (max-width: 760px) {                          ← «se o ecrã tiver menos de 760 px…»
  .event { grid-template-columns: 1fr; }              ← …os eventos passam a uma só coluna
  .event .thumb { order: -1; }                        ← …e a fotografia passa para cima do texto
}
```

As **media queries** (`@media`) são regras «se… então…» conforme o tamanho do ecrã. Mas a grelha «auto-adaptável» do primeiro exemplo evita muitas vezes escrevê-las: o rodapé funciona da mesma maneira (`minmax(180px, 1fr)`).

O menu no telemóvel

Em ecrã pequeno, o menu dobra-se atrás de um botão «Menu». O código está em `src/layouts/Base.astro` (botão `.menu-btn`) e o estilo em `global.css`.

Os pormenores que o tornam «profissional»

- `text-wrap: balance`: os títulos em duas linhas têm linhas de comprimento parecido.
- `@media (prefers-reduced-motion)`: tiram-se as animações a quem se sente incomodado.
- Uma largura máxima (`--max: 1440px`) para que o texto não se estique num ecrã gigante.

> **EXPERIMENTAR**
>
> F12 → ícone telemóvel/tablet («modo dispositivo») → escolha «iPhone»: veja o site reorganizar-se. Alargue a janela aos poucos e observe quando mudam as colunas.

9 · Fotografias e vídeos

As imagens são muitas vezes 80 % do peso de uma página. Uma fotografia de telemóvel pesa 4 a 8 MB; no site, a mesma fotografia pesa 50 a 200 KB. Como?

Três otimizações automáticas

1. **Ao enviar na administração**: a fotografia é reduzida a 2000 px no máximo e convertida em **WebP**, um formato mais leve do que JPEG (definição `transformations` em `src/admin/config.yml`).
2. **Ao construir o site**: o Astro cria **5 tamanhos** de cada fotografia (480, 800, 1200, 1600, 1920 px). Ficheiro: `src/components/Img.astro`.
3. **No navegador**: graças aos atributos `srcset` e `sizes`, um telemóvel descarrega a versão pequena e um ecrã grande a grande. E `loading="lazy"`: uma fotografia só é descarregada quando nos aproximamos dela ao percorrer a página.

O texto alternativo

Cada imagem tem um `alt`: uma descrição lida às pessoas cegas e usada pelo Google. P. ex. na página inicial: «Relaxamento final na shala do Evergreen Project» (editável na administração).

Os vídeos

- **YouTube / Vimeo** (recomendado): cola-se a ligação; o componente `Video.astro` só mostra uma miniatura e carrega o leitor ao clicar (versão «youtube-nocookie», sem cookies) → página rápida e respeitadora.
- **Ficheiro MP4**: um robô do GitHub (`.github/workflows/compress-videos.yml`) recomprime-o com a ferramenta **ffmpeg** (formato H.264, 1280 px máx.).

O visualizador em ecrã inteiro

Em `src/pages/[lang]/gallery.astro`, clicar numa fotografia abre-a em grande num elemento HTML nativo `<dialog>` — sem biblioteca externa: o navegador já trata da janela, da tecla Esc e do foco.

10 · Um site em 8 línguas

Grego (língua por defeito), inglês, francês, espanhol, italiano, russo, alemão, português. Três mecanismos:

1. **Os endereços** começam pela língua: `/el/`, `/pt/`... A pasta `[lang]` de `src/pages/` gera cada página em cada língua. A lista das línguas está em `src/i18n.ts`.
2. **O conteúdo**: cada ficheiro YAML tem uma parte por língua (capítulo 6).
3. **Os pequenos textos** («Menu», «Ler mais», «Adicionar à agenda»...): `src/content/settings/ui.yml`, 96 frases × 8 línguas.

A «reserva» (fallback)

Se uma frase não estiver traduzida numa língua, o site mostra o inglês, ou então o grego. Nunca há buracos. O código: função `localize` em `src/lib.ts`.

Dizer ao Google que existem traduções

No `<head>` de cada página (`Base.astro`), etiquetas `<link rel="alternate" hreflang="pt">` indicam as versões noutras línguas: um visitante português chega à página em português.

> **VER**
>
> No site, o seletor de língua no canto superior direito: mantém a mesma página e muda só o prefixo (`/pt/contact/` → `/de/contact/`).

11 · As funcionalidades, uma a uma

Para cada funcionalidade: o que faz, a ferramenta ou o serviço usado, e o ficheiro a abrir.

Funcionalidade	Como é feita	Onde ver
Calendário mensal	Um pequeno script JavaScript desenha o mês e pinta os dias ocupados; os eventos passados desaparecem sozinhos (comparando com a data de hoje).	`src/pages/[lang]/calendar.astro`, `src/components/EventList.astro`
«Adicionar à agenda» (Google)	Uma simples ligação para o Google Calendar com o título, as datas e o fuso «Europe/Athens»: o Google converte para a hora do visitante.	`src/calendar.ts` (função `googleUrl`)
Apple / Outlook (.ics)	Um ficheiro `.ics`: formato de texto normalizado (RFC 5545) que todas as agendas percebem. As horas são convertidas em tempo universal com a ferramenta `Intl` do JavaScript.	`src/calendar.ts` (função `ics`), `src/pages/[lang]/events/[id].ics.ts`
Subscrever a agenda	Um ficheiro `/calendar.ics` com todos os eventos; a agenda do visitante relê-o regularmente.	`src/pages/calendar.ics.ts`
Mapa	Um mapa **OpenStreetMap** (livre e gratuito) dentro de um `<iframe>`, mais uma ligação «Abrir nos mapas» para o Google Maps.	`src/pages/[lang]/[page].astro` (parte de contacto)
Formulário de contacto	O serviço **Web3Forms**: o formulário envia os campos para o servidor deles, que os reencaminha por e-mail. Uma caixa invisível apanha os robôs («honeypot»).	mesmo ficheiro, `<form action="https://api.web3forms.com/submit">`
Donativo / IBAN	O IBAN em grupos de 4 + um botão que o copia (API **Clipboard** do navegador).	mesmo ficheiro, bloco `donate` + `<script>` no fim
Histórias	Formulário → pequeno servidor (Worker, capítulo 13) → guardada «por publicar» → uma pessoa da administração valida.	`src/pages/[lang]/stories.astro`, `worker/index.js`
Galeria	Álbuns em «mosaico» CSS, visualizador `<dialog>`.	`src/pages/[lang]/gallery.astro`
Faixa de aviso	Se o campo «Ανακοίνωση» da administração estiver preenchido, aparece uma faixa laranja no topo de todas as páginas.	`Base.astro` (`site.announcement`)
Ligação aos estúdios de ioga	Uma ligação externa (sem duplicação: aulas e preços ficam no site deles).	`src/content/settings/site.yml`
Formatação dos textos	Os textos longos são escritos em **Markdown** (`**negrito**`, `- lista`, `[ligação](endereço)`), transformado em HTML pelo pacote **marked**.	`src/lib.ts` (função `md`)

12 · A área de administração (sem código)

A Katerina e o Kostas não vão editar ficheiros YAML. Por isso damos-lhes uma **interface** no endereço `/admin/`: formulários, botões, arrastar e largar fotografias. É um **CMS** (Content Management System = sistema de gestão de conteúdos).

O CMS escolhido: Sveltia CMS

- Gratuito e de código aberto, funciona inteiramente no navegador.
- Lê e escreve diretamente os ficheiros YAML do repositório: **sem base de dados**.
- Trata as 8 línguas lado a lado e comprime as fotografias ao enviá-las.

Como se configura

Apenas dois ficheiros:

- `public/admin/index.html`: carrega o programa Sveltia (versão fixa + impressão digital de segurança, ver capítulo 16).
- `src/admin/config.yml`: descreve **cada formulário**. Para cada campo: o nome, a etiqueta, o tipo («widget»).

```
- { name: start, label: Έναρξη · Start date, widget: datetime, type: date }   ← um calendário para escolher a data
- { name: price, label: Τιμή · Price, widget: string, i18n: true }          ← um campo de texto, traduzível
- { name: image, label: Φωτογραφία · Photo, widget: image }                ← um botão para enviar fotografia
- { name: published, label: Published, widget: boolean }                   ← uma caixa de seleção
```

O que acontece quando se clica em «Save»

Admin: Save → O ficheiro YAML muda no repositório → Um robô reconstrói o site → Online ~2 min depois

Cada gravação é um «commit»: pode-se sempre voltar a uma versão anterior.

A ajuda à escrita

Ao lado do campo que se está a editar aparece um botão **«✓ Correct»**: propõe uma versão corrigida (ortografia, gramática, pontuação), para aceitar ou não. E a cada gravação, os textos gregos novos ou alterados são **traduzidos automaticamente** para as outras 7 línguas. As duas coisas passam pelo Worker (capítulo 13), que chama um serviço de correção e tradução automáticas. Código: `public/admin/index.html` (o botão) e `worker/ai.js`.

> **VER**
>
> `docs/GUIDE-ADMIN.md`: o manual escrito para quem administra (grego + inglês). Um bom exemplo de documentação para pessoas não técnicas.

13 · O pequeno servidor: o Worker

Um site estático não consegue guardar segredos nem receber dados (não há nenhum programa a correr no servidor). Por isso, para algumas necessidades, juntamos um **servidor minúsculo** e gratuito na **Cloudflare**: um **Worker** (uma função JavaScript que corre quando é chamada).

Necessidade	O que o Worker faz
Acesso da administração	Mostra um formulário de e-mail + palavra-passe, verifica-o e entrega um «crachá» de sessão (válido 30 dias), assinado com uma chave secreta (assinatura HMAC) para não poder ser falsificado.
Gravações da administração	Passa os pedidos da administração ao armazenamento, juntando a verdadeira chave de acesso que só o Worker conhece.
Ajuda à escrita	Corrige um texto quando pedido; traduz cada gravação para as 7 línguas antes de a guardar.
Histórias	Recebe o formulário, filtra o spam, verifica que a fotografia é mesmo uma imagem, guarda a história «por publicar» e avisa a equipa.

Os «segredos» (palavras-passe da administração, chave de acesso) ficam guardados na Cloudflare, cifrados — **nunca** no código do repositório.

> **VER**
> `worker/index.js` (bem comentado, dividido em secções: acesso, retransmissão, histórias) · `worker/ai.js` (ajuda à escrita) · `worker/wrangler.toml` (definições) · `worker/README.md` (instalação passo a passo). A ferramenta **wrangler** (`npx wrangler deploy`) envia o Worker para a Cloudflare.

API: a palavra a conhecer

Uma **API** é uma «porta» que um programa abre a outros programas: envia-se um pedido para um endereço e recebe-se uma resposta. O site usa várias:

API	Usada para
API do GitHub	Ler / escrever os ficheiros de conteúdo (através do Worker)
Serviço de tradução / correção	Traduzir e corrigir os textos da administração (através do Worker)
Web3Forms	Enviar o formulário de contacto por e-mail
Cloudflare Turnstile (opcional)	Anti-robôs, uma alternativa respeitadora aos «CAPTCHA»
APIs do navegador	Área de transferência (copiar o IBAN), `Intl` (datas e fusos horários), `<dialog>`
Google Calendar (ligação)	Adicionar um evento à sua agenda

14 · Ser encontrado no Google (SEO)

O SEO (otimização para motores de busca) consiste em ajudar os motores de busca a perceber e a apresentar bem o site.

Elemento	Papel	Onde
`<title>` e `<meta name="description">`	O título azul e as duas linhas de texto nos resultados do Google.	`Base.astro`
Open Graph (`og:image`…)	A pré-visualização (fotografia + título) quando se partilha a ligação no Facebook, WhatsApp…	`Base.astro`
Mapa do site (`sitemap-index.xml`)	A lista de todas as páginas para o Google; gerado pelo **@astrojs/sitemap**.	`astro.config.mjs`
`robots.txt`	Diz aos robôs o que podem visitar (a administração não).	`src/pages/robots.txt.ts`
Dados estruturados (JSON-LD)	Uma ficha legível por máquinas: «isto é uma ONG, eis a morada, o telefone» (vocabulário **schema.org**).	`Base.astro` (`ld`)
`hreflang`	As traduções de cada página.	`Base.astro`
Endereços legíveis	`/pt/contact/` em vez de `/page?id=37`.	nomes dos ficheiros em `src/pages/`

EXPERIMENTAR

Cole o endereço do site em *opengraph.xyz* para ver a pré-visualização de partilha, ou procure o «Teste de resultados avançados» do Google para ver a ficha JSON-LD.

15 · Acessibilidade: um site para todos

Cerca de uma pessoa em cada cinco tem uma deficiência (visão, mobilidade, audição, atenção…). Um site acessível é também mais agradável para todos e melhor classificado pelo Google.

- **Textos alternativos** nas imagens (`alt`).
- **Contrastes** suficientes: texto cinzento-claro sobre fundo creme é ilegível para muita gente. O crédito no rodapé foi escurecido por isso.
- **Títulos por ordem** (`h1` e depois `h2`…): os leitores de ecrã usam-nos como índice.
- **Ligação «Saltar para o conteúdo»** escondida, visível com o teclado (`.skip` em `Base.astro`): evita atravessar o menu todo.
- **Navegação por teclado**: contorno visível no elemento selecionado (`:focus-visible` em `global.css`).
- **Nomes claros** para botões e ligações sem texto (`aria-label`): p. ex. as fotografias-ligação da página inicial.
- Respeito pela definição «reduzir movimento» do telemóvel.

> **EXPERIMENTAR**
>
> No site, carregue várias vezes na tecla **Tab**: deve ver sempre onde está. Depois F12 → separador **Lighthouse** → «Acessibilidade» → Analisar.

16 · Segurança

Risco	Proteção neste site	Onde
Alguém injetar um script numa página	**CSP** (Content-Security-Policy): uma lista branca do que a página pode carregar; o navegador bloqueia tudo o resto.	`astro.config.mjs` (`security.csp`)
Um serviço externo (o CMS) ser alterado sem sabermos	**Versão fixa + SRI**: uma impressão digital do ficheiro; se mudar um único byte, o navegador recusa-se a executá-lo.	`public/admin/index.html` (`integrity="sha384-…"`)
Fuga de palavras-passe / chaves	Segredos só na Cloudflare, nunca no repositório; ficheiros de segredos locais ignorados pelo Git.	`.gitignore`, `worker/README.md`
Tentativas de palavra-passe em rajada, spam	Limite de 3 tentativas / minuto / visitante, campo armadilha, tempo mínimo, Turnstile opcional.	`worker/index.js`, `wrangler.toml`
Envio de ficheiros falsos	Verificação dos primeiros bytes (a «assinatura»): um JPEG começa sempre por `FF D8 FF`.	`worker/index.js` (`looksLike`)
Espionagem da ligação	HTTPS em todo o lado (o cadeado).	alojamento
O site mostrado dentro de uma moldura armadilhada	Cabeçalho `X-Frame-Options`.	`public/_headers`
Rastreio dos visitantes	Sem cookies, sem ferramentas de estatística, tipos de letra e vídeos sem rastreadores.	página de Privacidade

PARA DIZER

«A melhor segurança é ter pouco para proteger: um site estático não tem base de dados nem palavras-passe para roubar. Os poucos segredos estão guardados num só sítio.»

17 · Rapidez (desempenho)

Um site lento afasta as pessoas (e o Google penaliza-o). As receitas aplicadas aqui:

- **Páginas já prontas** (estáticas): nada para calcular durante a visita.
- **Imagens**: WebP, vários tamanhos, carregamento diferido (capítulo 9). A primeira fotografia grande, pelo contrário, é carregada com prioridade (`fetchpriority="high"`).
- **Muito pouco JavaScript**: só pequenos scripts onde são úteis. O Astro não envia JavaScript por defeito.
- **Tipos de letra alojados no site** em vez do Google Fonts.
- **Vídeos carregados ao clicar**.
- **Cache**: os ficheiros cujo nome contém uma impressão digital (pasta `_astro/`) ficam guardados no navegador durante um ano (`public/_headers`).

Resultado medido com o **Lighthouse** (integrado no Chrome, separador «Lighthouse») em telemóvel: desempenho 98–99/100, página inicial ≈ 256 KB.

Os indicadores a conhecer

Indicador	Significado	Objetivo
LCP	Tempo até mostrar o maior elemento (muitas vezes a fotografia principal)	< 2,5 s
CLS	A página «salta» durante o carregamento?	< 0,1
INP / TBT	A página responde depressa aos cliques?	o mais baixo possível

18 · Testar o site

Os testes automáticos

Pequenos programas verificam que o código faz o que deve. Aqui com a ferramenta integrada no Node (`node:test`), lançados com `npm test`:

- `tests/calendar.test.ts`: «10 h em Atenas em dezembro = 8 h em tempo universal, mas 7 h em julho (hora de verão)», «o ficheiro .ics está bem formado»…
- `tests/worker.test.ts`: «uma palavra-passe errada é recusada», «um ficheiro de imagem falso é rejeitado», «a chave secreta nunca chega ao navegador»…
- `tests/ai.test.ts`: «só o grego novo ou alterado é traduzido», «uma tradução escrita à mão é mantida»…

As verificações manuais (a cada grande alteração)

O quê	Como
Apresentação	Num telemóvel verdadeiro + o modo dispositivo das ferramentas de programador; Chrome, Safari, Firefox.
Ligações partidas	Clicar em todo o lado, ou uma ferramenta de verificação de ligações.
Acessibilidade	Lighthouse, a extensão **axe DevTools**, navegação por teclado.
Erros	F12 → separador «Consola»: nenhuma linha vermelha.
Conteúdo	Reler em cada língua (idealmente por falantes nativos).

19 · Publicar: alojamento, domínio, DNS

O alojamento

A pasta `dist/` feita por `npm run build` é colocada num **alojamento** (hosting). Para um site estático chegam serviços gratuitos: **GitHub Pages** (usado hoje), **Cloudflare Pages** (previsto a seguir), Netlify…

O robô de publicação

`.github/workflows/deploy.yml` é uma receita que o GitHub executa automaticamente a cada alteração em `main`: instalar o Node → `npm ci` → `npm run build` → publicar `dist/`. É a **integração / implementação contínuas** (CI/CD): ninguém precisa de «pôr online» à mão.

O nome de domínio

Um nome como `evergreenproject.gr` compra-se (≈ 10–15 €/ano) a um **registador** (Papaki, DNHOST, OVH…). Aluga-se ao ano, não é nosso: atenção à renovação!

O DNS: a lista telefónica da Internet

O DNS traduz um nome (`evergreenproject.gr`) no endereço de um servidor. Juntam-se-lhe «registos»:

- **A**: «este nome aponta para este servidor (endereço IP)».
- **CNAME**: «este nome é um pseudónimo deste outro nome» (p. ex. `www` → o alojamento).
- **MX**: «os e-mails deste domínio vão para aqui».

> **VER**
>
> Em `README.md`, as secções sobre o domínio próprio e sobre deixar o GitHub: os passos concretos para ligar um domínio e mudar de alojamento.

20 · O ciclo de vida de uma alteração

Para quem programa (mudar o design ou uma funcionalidade)

1. Criar um **ramo** (cópia de trabalho).
2. Alterar os ficheiros no VS Code; verificar em direto com `npm run dev`.
3. Correr os testes (`npm test`) e a construção (`npm run build`).
4. **Commit** com uma mensagem clara, **push** para o GitHub.
5. Abrir um **pull request**, rever e depois **juntar** ao `main`.
6. O robô publica: ~2 minutos depois, toda a gente vê.

Para quem administra (mudar um texto, uma data, uma fotografia)

1. Abrir `/admin/` e entrar.
2. Alterar e clicar em **Save**.
3. ~2 minutos depois está online. Nada para instalar.

> **EXEMPLO REAL**
>
> No histórico do repositório («Commits»), procure «Inhabitants: new photo of our dogs»: uma fotografia nova em `src/assets/uploads/` e uma única linha alterada em `src/content/inhabitants/dogs.yml` — exatamente o que a administração faz quando se muda uma fotografia.

21 · Por onde começar a aprender

Um percurso sugerido (cerca de 3 meses a algumas horas por semana)

Etapa	Objetivo	Recursos gratuitos
1. HTML	Fazer uma página com títulos, textos, imagens, ligações	MDN Web Docs («Aprender desenvolvimento web»), freeCodeCamp
2. CSS	Cores, tipos de letra, depois Flexbox e Grid	Os jogos **Flexbox Froggy** e **Grid Garden**, web.dev/learn/css
3. Design adaptável	Uma página que se adapta ao telemóvel	web.dev/learn/design
4. Git e GitHub	Guardar e publicar o seu trabalho	GitHub Skills, «Learn Git Branching»
5. JavaScript	Juntar um pouco de interação (botão, menu)	javascript.info, freeCodeCamp
6. Um gerador	Refazer um pequeno site com Astro	docs.astro.build (tutorial «Construir um blogue»)
7. Publicar	Pôr o seu site online de graça	GitHub Pages, Cloudflare Pages

Pequenos exercícios com este site (sem risco, no seu computador)

1. Mudar a cor `--clay` em `src/styles/global.css` e observar.
2. Juntar um evento de teste: copiar um ficheiro de `src/content/events/`, mudar as datas e o título.
3. Mudar o texto de um botão em `src/content/settings/ui.yml`.
4. Juntar um habitante (um animal!) em `src/content/inhabitants/`.
5. Ler `src/components/Story.astro` e tentar criar o seu próprio componente.

> **CONSELHO FINAL**
>
> «Não se aprende a ler, aprende-se a estragar e a consertar. Com o Git nunca nada fica realmente estragado: pode-se sempre voltar atrás.»

22 · Glossário

API	Porta de acesso que um serviço abre a outros programas.
Build	A construção do site final (pasta `dist/`).
Cache	Cópia guardada pelo navegador para não voltar a descarregar.
CI/CD	Robôs que testam e publicam automaticamente.
CMS	Interface para gerir o conteúdo sem programar (aqui o Sveltia, em `/admin/`).
Commit	Uma alteração guardada no histórico, com uma mensagem.
Componente	Peça de página reutilizável (`Img`, `Story`…).
CSP	Lista branca de segurança do que uma página pode carregar.
DNS	A lista que traduz um nome de domínio no endereço de um servidor.
Repositório (repo)	A pasta do projeto com todo o seu histórico (no GitHub).
Front-end / back-end	O que corre do lado do visitante / no servidor.
Alojamento	Serviço que mantém os ficheiros do site online 24 h/24.
i18n	«Internacionalização» (i + 18 letras + n): gestão das línguas.
Layout	A moldura comum a todas as páginas.
Markdown	Forma simples de escrever texto formatado (`**negrito**`).
npm / pacote	A loja de ferramentas JavaScript / uma ferramenta descarregada de lá.
Pull request	Pedido para juntar um ramo, com revisão.
Responsive	Que se adapta a todos os tamanhos de ecrã.
SEO	Ser bem encontrado e apresentado pelo Google.
SRI	Impressão digital que garante que um ficheiro externo não foi alterado.
Estático	Páginas feitas antecipadamente e não a cada visita.
URL	O endereço de uma página.
WebP	Formato de imagem moderno, mais leve do que JPEG.
Worker	Pequena função de servidor alojada na Cloudflare.
YAML	Formato de texto legível («chave: valor») que guarda o conteúdo.